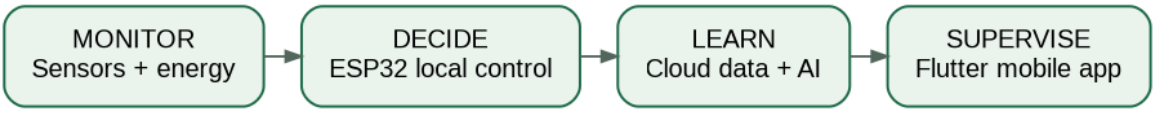
IRRIGA-TECH

SYSTEM ARCHITECTURE & IMPLEMENTATION BLUEPRINT

AI-driven solar-powered IoT smart greenhouse
with intelligent crop nutrition and energy-aware irrigation



Prepared as an engineering implementation baseline

Version 1.0 | 12 October 2026 | Status: Draft for technical review

Document purpose and design status

This blueprint translates the uploaded Irriga-Tech proposal into an implementable architecture for a 3 m × 4 m greenhouse prototype. It defines subsystem responsibilities, interfaces, local control algorithms, cloud and mobile integration, data handling, safety, testing, and a staged delivery plan.

Design rule The ESP32 is the safety authority for physical outputs. Cloud AI may advise or propose a schedule, but it must never bypass local interlocks, pump timeouts, sensor validation, or the crop-critical irrigation override.

Contents

- 1. Executive architecture decisions
- 2. Scope, assumptions and engineering clarifications
- 3. Logical architecture and subsystem responsibilities
- 4. End-to-end system workflow
- 5. Hardware and electrical integration
- 6. Software architecture and interfaces
- 7. Data model and API contract
- 8. Control and AI algorithms
- 9. Arduino C++ firmware design
- 10. Failure handling and cybersecurity
- 11. Verification and performance evaluation
- 12. Delivery plan, risks and acceptance gates
- Appendix A. Reference implementation patterns
- Appendix B. Technical references

How to use this blueprint

Treat values such as sample intervals, threshold percentages, timeouts and state-of-charge limits as proposed configuration defaults, not final agronomic or electrical values. Confirm them after selecting the exact ESP32 board, sensors, battery chemistry, pump, crop, growing medium and irrigation hardware. The uploaded proposal specifies the general system concept; it does not specify every wiring detail, cloud endpoint, data field or AI training dataset.

1. Executive architecture decisions

Decision	Recommended baseline	Why
Edge versus cloud	Local-first controller on ESP32; cloud for history, forecasts, AI and remote visibility.	Keeps essential control operating during network or cloud outages.
Device firmware	Arduino framework with C++ on ESP32. Separate sensor, actuator, safety, irrigation, climate and communications modules.	Easier to test, maintain and debug than one large loop.
Cloud entry point	Authenticated HTTPS API handled by a Python Cloud Function or Cloud Run service.	Validates device identity and payloads before writing to the database.
Data store	Firebase Cloud Firestore for prototype configuration, telemetry summaries, events, commands and recommendations.	Consistent app/cloud model; event volume and cost should be monitored.
AI service	Python service; start with data validation and agronomy-reviewed baseline. Add ML only when reliable training targets exist.	Avoids presenting unvalidated rule output as trained AI.
Mobile app	Flutter + Firebase Authentication + authorised database access.	Provides login, dashboards, alerts, reports and controlled manual overrides.

Decision	Recommended baseline	Why
Command model	Commands have IDs, expiry times, allowed action types and acknowledgements.	Prevents stale remote commands from unexpectedly actuating equipment.
Safety priority	Electrical faults, invalid sensors, empty tank, maximum runtime and local crop-safety limits override energy optimisation.	Protects equipment and crops even if the cloud recommendation is wrong.

Core principle

Irriga-Tech should be implemented as a dependable automation system first and an increasingly predictive system second. A deterministic controller can provide real value before a machine-learning model has enough observations to be evaluated credibly.

2. Scope, assumptions and engineering clarifications

2.1 In-scope capabilities

- 3 m × 4 m greenhouse prototype; 4 irrigation zones are proposed in the bill of materials.
- Environmental and water monitoring: soil moisture, temperature, relative humidity, light and tank level.
- Energy monitoring: PV voltage/current/power and battery voltage/current, with a documented state-of-charge estimate.
- Automatic irrigation using locally validated rules, zone selection, run-time limit, dry-run protection and safe crop-critical override.
- Greenhouse climate support: ventilation and motorised shade control using configured setpoints and hysteresis.
- Cloud storage, historical charts, alerts, weather/solar forecast ingestion and mobile remote supervision.
- Crop-nutrition recommendations, initially advisory and reviewed against trusted agronomic guidance.

2.2 Explicit non-goals for the first prototype

- Automatic fertilizer mixing or dosing before recommendation logic has been validated by an agronomist.
- Unsupervised online model retraining directly from every new observation.
- Guaranteeing yield increases or water savings before controlled measurement.
- Operating a pump solely from an internet command without local checks.
- Using soil moisture, ambient humidity and light alone as definitive measurements of N-P-K deficiency.

2.3 Items to resolve before procurement

Proposal discrepancy / gap	Action required
Hardware list describes a 200 W panel and 20 A controller, while the budget/BOM describe a 250 W panel and 30 A controller.	Select one design after calculating daily Wh, peak concurrent load, PV yield, battery reserve, controller voltage/current limits and cable protection.
SoC is named, but voltage/current sensors alone may not provide accurate battery SoC.	Document battery chemistry and SoC method. Use a suitable battery monitor/fuel gauge or validated coulomb-counting approach; treat voltage-only SoC as an estimate.
The component list includes LDRs but no direct pH, EC or nutrient sensor.	For fertilizer AI, add soil/growing-medium test data and agronomist-verified labels; decide whether pH/EC measurements are included in the project budget.
The exact ESP32 board, pump current, actuator voltage, relay/driver ratings and sensor part numbers are not finalised.	Freeze the part numbers before pin mapping, PCB design and code commissioning.
Budget arithmetic needs reconciliation. Listed items add to KES 310,600 before contingency; 10% is KES 31,060; calculated total is KES 341,660.	Recheck the source budget line items and confirm any omitted item before approval; proposal states KES 348,260.

Important These are design-review findings. Do not silently purchase parts based on conflicting specifications; issue a revised, signed bill of materials after power and electrical calculations.

3. Logical architecture and subsystem responsibilities

Figure 1 shows the recommended separation between greenhouse equipment, embedded control, cloud services, forecast providers and the mobile application.

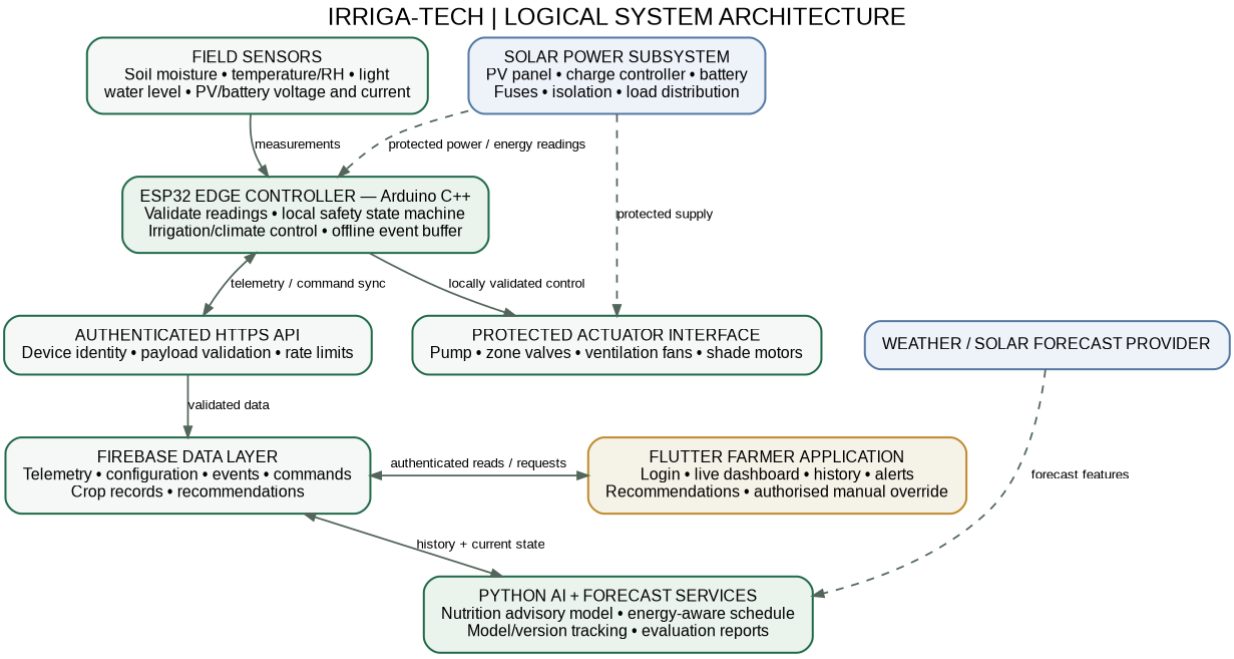


Figure 1. Logical architecture and trust boundaries.

3.1 Subsystem responsibilities

Subsystem	Responsibilities	Must not do
Sensors and measurement	Acquire calibrated signals; expose units, validity and timestamp/age.	Do not treat missing, stale or out-of-range readings as zero.
ESP32 edge control	Schedule sampling; validate signals; control actuators; implement safety interlocks and offline fallback.	Do not depend on cloud round trips for emergency pump stop or maximum runtime.
Actuator/power interface	Drive pump/valves/fans/servos through correctly rated drivers, fuses and power supplies.	Do not power motors or pumps directly from ESP32 GPIO pins.
Cloud API	Authenticate device; validate payloads; enforce rate/size limits; persist telemetry/events.	Do not accept arbitrary GPIO values or unsigned high-privilege commands.
Data layer	Store configuration, telemetry, event history, crop/fertilizer records, recommendations and command state.	Do not grant unrestricted database writes to unauthenticated clients.
AI/forecast service	Feature preparation; forecast retrieval; model inference; recommendation versioning and evaluation.	Do not autonomously override the local safety policy.
Flutter application	Authenticated dashboards, crop records, charts, alerts, recommendation review and authorised overrides.	Do not represent a command as executed until device acknowledgement confirms it.

4. End-to-end system workflow



Figure 2. Operating cycle. Local safety decisions run even when the cloud branch is unavailable.

4.1 Normal operating cycle

1. ESP32 boots with pump, valves, fans and shade actuators in known safe states; self-checks configuration and communication status.
2. At the configured sample interval, sensors are read. Raw values are converted to calibrated engineering units and checked for plausibility and freshness.
3. Local safety rules evaluate sensor faults, tank availability, pump timeout, battery protection and crop-specific moisture limits.
4. The irrigation state machine decides whether to keep irrigation off, start a zone, wait for an energy window, or raise a fault/alert.
5. Climate control separately evaluates heat/light readings and controls fans/shade using hysteresis, movement limits and feedback where available.
6. Each significant action produces an event containing timestamp, decision reason, input values, selected zone, duration and result.
7. When connected, the device uploads telemetry and event batches to an authenticated API. Cloud processing persists data and may return validated configuration or an expiring command.
8. Python services compute recommendations and forecast-aware advice; Flutter reads authorised data and displays the actual device state, not merely requested actions.
9. On next cycle, the ESP32 re-validates everything. Cloud advice never bypasses local limits.

4.2 Offline behaviour

If Wi-Fi, API or cloud database is unavailable, the ESP32 continues sensor acquisition and local control using the last validated configuration. It buffers a bounded number of events if memory allows, drops the oldest non-critical telemetry if the buffer is full, and logs an explicit buffer-overflow/fault counter. It does not accept unverified commands or keep a pump running while waiting for the cloud.

5. Hardware and electrical integration

5.1 Recommended functional blocks

Block	Prototype implementation guidance
Controller	ESP32 development board; confirm exact module and ADC/pin map. Use a watchdog and non-blocking scheduling with millis()-based timers.
Soil sensing	One calibrated capacitive moisture sensor per independently controlled bed/zone is a reasonable starting arrangement; calibrate in the actual growing medium.
Temperature / humidity	DHT22 as proposed; mount away from direct sun, wet surfaces and hot electronics. Validate its operating range and reading stability.
Light sensing	LDR as proposed can support simple threshold control after calibration. If measured illuminance is an evaluation variable, consider a calibrated digital light sensor.
Water level	Ultrasonic sensor can estimate tank level; mount with suitable splash protection and filter readings. A separate low-level float switch can provide a robust low-water interlock.
Battery/PV metering	Measure voltage and current with correctly rated, calibrated devices; compute P(W)=V(V)×I(A). Document measurement point and sample interval.
Pump/valves/fans	Use rated DC MOSFET drivers or suitable relays, flyback protection for inductive loads, proper fuses and separate power distribution. Verify inrush/stall current.
Servos	Use a suitably rated separate supply and common reference where required; do not power servo motors from the ESP32 3.3 V pin.
Solar subsystem	PV panel, matching charge controller, battery with suitable enclosure/isolator, fuses, cable sizing and charge/battery protection.
Enclosure/wiring	Separate wet plumbing from electronics; use glands, strain relief, labelled terminals, drip loops and enclosure protection suitable for the installation environment.

5.2 Example pin map — provisional only

The following is an example for a classic ESP32-WROOM-32 DevKit, not a final wiring instruction. Different ESP32 variants have different pin availability. Avoid pin assignment until the exact board and sensor breakout boards are confirmed. For the classic ESP32, use ADC1 pins for analogue moisture/light inputs when Wi-Fi is active; check the board/module pinout and electrical limits.

Signal	Example GPIO	Engineering note
Soil moisture zones 1–4	32, 33, 34, 35	ADC1 candidates on classic ESP32; GPIO34/35 are input-only.
LDR analogue input	36 or 39	ADC1 input-only candidates on classic ESP32.
I²C energy monitors	21 SDA, 22 SCL	Shared bus; check addresses, pull-ups and sensor voltage compatibility.
Pump enable	18	Drive through a rated driver/relay interface; choose a defined OFF state during boot.
Valve/fan outputs	Use remaining verified outputs or an I/O expander	The number of outputs can exceed convenient GPIO allocation; avoid boot-strapping pin surprises.
Ultrasonic trigger/echo	Choose verified digital GPIOs	If the module echo output is 5 V, level-shift/divide it to a safe ESP32 input voltage.
Shade servos	PWM-capable verified GPIOs or I²C PWM driver	Separate servo supply; add end-stop/position feedback if required for robust operation.

Hardware safety gate Test all output drivers with the pump disconnected first. Confirm the relay/MOSFET default state, emergency cutoff, tank-empty interlock, fuse, flyback protection and maximum runtime before connecting water.

6. Software architecture and interfaces

6.1 ESP32 firmware modules (C++)

Module / file	Primary responsibility
main.ino / app_main.cpp	Initialise modules; schedule tasks; handle boot, watchdog and state transitions.
config.h / ConfigManager	Validated crop profile, thresholds, run-time limits, pin configuration and firmware version.
SensorManager	Read raw sensors; calibration; median/EMA filtering; range/freshness checks; sample timestamp.
EnergyManager	PV power, load estimates, battery monitoring, reserve rules and energy-availability status.
SafetyManager	Centralised pump-off causes, stale-sensor detection, dry-run prevention, hard timeouts and fault latching.
IrrigationController	Zone sequencing; moisture hysteresis; energy-aware wait; critical crop override; cooldown.
ClimateController	Fan and shade decisions with hysteresis, deadbands, run-time/motion limits and safe defaults.
NetworkClient	Wi-Fi reconnect with backoff; HTTPS telemetry; bounded retry/offline buffer; command polling.
CommandValidator	Allowed command list, command ID, timestamp, expiry, role/authority and local-limit checks.
EventLogger	Record state changes, irrigation decisions, sensor faults, remote commands and actuator outcomes.

6.2 Cloud/API services

Endpoint / service	Purpose	Expected response

Endpoint / service	Purpose	Expected response
POST /api/v1/telemetry	Authenticated device submits validated sensor snapshot.	Accepted/rejected, server timestamp, next upload interval.
POST /api/v1/events	Upload fault/irrigation/actuator events, optionally batched.	Accepted event IDs or validation errors.
GET /api/v1/device-config	Fetch latest approved config version.	Versioned thresholds and non-secret configuration.
GET /api/v1/commands?after=...	Fetch pending commands newer than device cursor.	Only commands for this device, each with ID and expiresAt.
POST /api/v1/commands/{id}/ack	Acknowledge command accepted/rejected/completed.	Persisted status and server timestamp.
Python recommendation job	Compute advisory nutrient recommendation / energy schedule from validated features.	Versioned recommendation with model/version, evidence, confidence and review status.
Flutter / Firebase SDK	Authenticate users and read their authorised greenhouse data.	Live UI state plus clear stale/offline indicators.

A simple prototype can use HTTPS uploads every 30–60 seconds and command polling every 5–15 seconds as starting configuration values, not fixed requirements. Tune the intervals based on data resolution, network availability, energy use and cloud costs. Actuator timeouts and safety must operate much faster locally.

6.3 Command contract

Remote commands should request high-level actions such as SET_IRRIGATION_MODE, START_ZONE, STOP_IRRIGATION, SET_SHADE_TARGET or SET_CONFIG_VERSION. Do not allow the app/cloud to specify arbitrary GPIO numbers or directly set a pump pin. START_ZONE should include zone ID, maximum duration and command expiry; the ESP32 enforces a stricter local duration limit if the request exceeds its configured maximum.

7. Data model and API contract

7.1 Recommended top-level data entities

Entity / collection	Key fields	Purpose
devices/{deviceId}	siteId, firmwareVersion, lastSeenAt, configVersion, healthStatus	Device identity and operational health.
greenhouses/{greenhouseId}	location metadata, crop profile, time zone, owner IDs	Greenhouse and access scope.
telemetry/{readingId}	deviceId, timestamp, moistureByZone, temperatureC, humidityPct, light, tankLevel, pvV, pvA, batteryV, batteryA, estimatedSoC	Time-series snapshot or batched readings.
events/{eventId}	timestamp, type, severity, decisionReason, inputs, outputs, result	Auditable record of irrigation, faults and actions.
cropProfiles/{cropProfileId}	crop, variety, growthStage, moisture bounds, max irrigation delay, light/temp rules, validatedBy	Versioned crop-specific configuration.
fertilizerApplications/{applicationId}	cropCycleId, product, composition, amount, unit, area, date, stage, operator, notes	Training/analysis record; no unreviewed auto-dosing.
recommendations/{recommendationId}	type, generatedAt, modelVersion, inputsSummary, recommendation, confidence/evidence, reviewerStatus, expiry	Traceable AI/advisory outputs.
commands/{commandId}	deviceId, type, args, createdAt, expiresAt, requestedBy, status, ackAt, result	Remote control queue with audit trail.

7.2 Telemetry example

```
{
  "schemaVersion": 1,
  "deviceId": "GH001-ESP32-01",
  "sequence": 1842,
  "deviceUptimeMs": 953204,
  "observedAt": "2026-10-12T09:30:00Z",
  "sensors": {
    "soilMoisturePct": {"zone1": 42.5, "zone2": 44.1},
    "temperatureC": 28.4,
    "relativeHumidityPct": 67.0,
    "lightRaw": 720,
    "tankLevelPct": 76.0
  },
  "energy": {
    "pvVoltageV": 18.2,
    "pvCurrentA": 6.5,
    "pvPowerW": 118.3,
    "batteryVoltageV": 12.4,
    "batteryCurrentA": 2.1,
    "estimatedSoCPct": 64,
    "socMethod": "calibrated_estimate"
  },
  "actuators": {"pump": false, "activeZone": null, "fans": true, "shadePct": 20},
  "health": {"wifi": true, "sensorFaults": [], "configVersion": 3}
}
```

This payload is illustrative. Use the units and measurement points you actually implement. Store both a device-observed time and a server-received time; sequence numbers help identify missing or duplicated uploads.

7.3 Data-quality rules

- Every record carries schemaVersion, deviceId, observedAt, device sequence number and server receivedAt.
- Missing sensors are represented as null/invalid plus a fault code, never as a meaningful zero.
- Reject out-of-range values and unexpectedly large payloads at the API boundary; preserve raw and calibrated values when useful.
- Keep units explicit: °C, %, V, A, W, L or tank-level %, seconds, and fertilizer amount plus unit/area basis.
- Recommendations include model/rules version, inputs used, generation time, supporting evidence and reviewer state.
- Command updates are audited and idempotent: a repeated command ID must not trigger a second watering event.

8. Control and AI algorithms

8.1 Algorithm A — sensor acquisition and validation

```
EVERY SAMPLE_INTERVAL:
  snapshot = read_all_sensors()
  for each sensor in snapshot:
    if electrical_reading_out_of_range(sensor): mark INVALID
    else calibrated = convert_to_engineering_units(sensor)
    filtered = median_or_ema(calibrated, recent_window)
    if no_recent_valid_sample(sensor): mark STALE

  if critical_sensor_invalid OR controller_self_test_failed:
    enter FAULT_SAFE
    set pump OFF; close/disable unsafe outputs; record fault
  else:
    mark snapshot VALID
    continue to irrigation and climate decision
```

Use a median filter for occasional spikes and/or an exponential moving average for smoothing. Filtering must not hide fast safety events; use separate raw-limit checks for critical signals. Sensor-specific calibration constants must be stored with a version.

8.2 Algorithm B — energy-aware irrigation priority

```
INPUT: valid soil moisture per zone, crop profile, tank state, battery reserve,
       PV power, predicted solar window, pump/driver status, last irrigation event

1. If emergency-stop active, sensor fault, over-current/over-temperature fault,
   or pump runtime limit exceeded: STOP pump; enter FAULT_SAFE.
2. If tank level is below LOW_WATER_LIMIT: STOP pump; create WATER_LOW alert.
3. If moisture is at/above the zone's MOISTURE_STOP threshold: keep pump OFF.
4. Estimate urgency from moisture deficit, rate of drying, crop stage and time
   already deferred. Use agronomist-reviewed thresholds.
5. If urgency is CRITICAL:
      irrigate only if electrical/driver safety allows and water is present;
      bypass energy-saving delay but never bypass electrical safety.
6. Else if battery SoC >= SOC_START and energy/current limits are satisfied:
      if PV power is adequate, start the selected zone now;
      otherwise compare predicted solar window with MAX_SAFE_DELAY.
7. If a safe solar window occurs before MAX_SAFE_DELAY:
      enter WAIT_FOR_ENERGY; keep pump OFF; schedule re-check; notify if delayed.
8. If no safe window occurs before MAX_SAFE_DELAY:
      choose the safest allowed watering action under crop and electrical limits;
      raise an exception/alert and record the reason.
9. While irrigating: monitor runtime, tank, current/flow if available, moisture,
   and stop conditions. Turn pump OFF first on any hard fault; de-energise valves
   in the safe order defined by the hydraulic design.
10. Record start/stop readings, litres if measured, energy consumed, zone, cause,
    config version and outcome. Apply a cooldown/hysteresis before re-triggering.
```

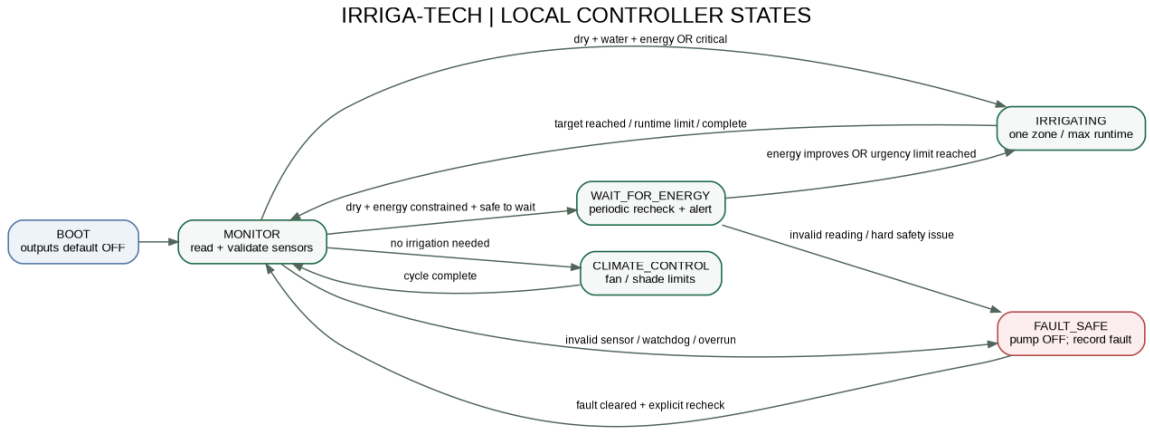


Figure 3. Local controller state machine; every transition remains subject to the central safety manager.

Priority order Safety fault > no water / equipment protection > crop-critical water demand > ordinary moisture demand > energy optimisation > convenience. Never allow the forecast to overrule a crop-critical deadline or a safety interlock.

8.3 Thresholds and hysteresis

Do not use a single threshold to start and stop the pump. Define at least MOISTURE_START and MOISTURE_STOP with a gap between them. For example, a demonstration profile might start near 35% and stop near 48%, but these are placeholders only: actual values must be calibrated for the crop, growing medium, sensor and sensor placement. Likewise, SOC_START, SOC_CRITICAL and MAX_SAFE_DELAY require battery and crop review.

8.4 Zone scheduling

1. Calculate a per-zone need score using moisture deficit, crop stage, drying trend and time since last watering.
2. Choose the highest-urgency eligible zone, subject to daily water budget and hydraulic limits.
3. Open the selected zone valve using the approved sequence; confirm expected actuator feedback if installed; run pump with a hard maximum runtime.
4. Stop on stop-threshold reached, water low, driver fault, excessive current, flow fault or timeout.
5. Use a cooldown and log the reason. Avoid watering multiple zones concurrently unless pump capacity, flow, pressure and energy budget are verified.

8.5 Algorithm C — ventilation and shade

```
IF temperature >= FAN_ON_TEMP for N consecutive valid samples:
    fans ON
ELSE IF temperature <= FAN_OFF_TEMP for N consecutive valid samples:
    fans OFF

IF light >= SHADE_CLOSE_LUX for N consecutive valid samples:
    move shade toward CLOSED, within travel/runtime limits
ELSE IF light <= SHADE_OPEN_LUX for N consecutive valid samples:
    move shade toward OPEN, within travel/runtime limits

Always enforce a deadband (ON threshold differs from OFF threshold),
maximum motor run time, minimum dwell time, manual override expiry, and
sensor-fault safe action. Lux values require calibration if an LDR is used.
```

The values FAN_ON_TEMP, FAN_OFF_TEMP, SHADE_CLOSE_LUX and SHADE_OPEN_LUX belong in a versioned crop/greenhouse profile, not hard-coded across the codebase. Include a maximum actuation time and use feedback/end-stops for shade movement if available.

8.6 Algorithm D — AI crop-nutrition pipeline

1. Data acquisition: combine crop species/variety/stage, substrate/soil test data, fertiliser product and amount, irrigation history, environment, dates and crop outcomes.
2. Quality control: validate units, missing values, duplicates, growth-cycle identity and records entered by an authorised operator.
3. Label definition: have an agronomist define the target, for example a validated nutrient/rate recommendation class or expected response metric. Avoid treating a yield outcome alone as proof that one fertiliser caused it.
4. Baseline first: implement a documented expert-reviewed decision table or retrieval of approved guidance while gathering real records. Label this baseline honestly; do not call it a trained ML model.
5. Model only with evidence: train a suitable supervised model if there is enough representative, correctly labelled data. Keep training, validation and test data separated by crop cycle or time to reduce leakage.
6. Evaluate: report suitable metrics for the target (e.g., MAE/RMSE for a numeric prediction; precision/recall and confusion matrix for classes), compare with baseline and expert review, and document confidence/limits.
7. Guardrails: recommendations must state evidence, assumptions, dose units and expiry; keep a human review/approval step and block out-of-range recommendations.
8. Feedback: store the farmer/expert decision and subsequent crop outcome, but retrain only in a controlled, versioned process with re-evaluation and rollback.

8.7 Cloud forecast and scheduling

A forecast provider can supply weather and solar-radiation features. Convert radiation/forecast variables into an estimated solar availability window only after considering panel size, tilt/orientation, system losses and battery charging characteristics. The

forecast is an input to the local scheduler, not an actuator command. On stale or missing forecasts, fall back to current PV power, battery reserve and the deterministic local policy.

9. Arduino C++ firmware design

9.1 Suggested repository structure

```
irrigatech-esp32/
  platformio.ini               # optional; Arduino IDE project is also acceptable
  src/ or IrrigaTech.ino
  include/
    Config.h
    DataTypes.h
    SensorManager.h
    EnergyManager.h
    SafetyManager.h
    IrrigationController.h
    ClimateController.h
    NetworkClient.h
    EventLogger.h
  test/
    test_irrigation_policy.cpp
    test_payload_validation.cpp
  docs/
    wiring-register.md
    calibration-records.md
```

9.2 Coding rules

- Use non-blocking millis()-based schedules for sampling, network retry and actuator timeouts. Avoid long delay() calls in the control loop.
- Set every output to a known safe state immediately at boot before network initialisation.
- Centralise every pump stop path in SafetyManager so fault handling is consistent.
- Separate pure decision functions from GPIO and network I/O; pure functions are easier to unit test.
- Use enums/structs for states and readings; avoid scattered magic numbers.
- Use a watchdog and bounded buffers; keep network calls time-bounded so control cycles cannot hang.
- Persist configuration version and last-known state, but never resume an interrupted watering run blindly after reboot.
- Log command ID and decision reason. Acknowledge remote commands only after local validation and report executed state separately.

9.3 Reference C++ control pattern

The following is an architecture pattern, not a complete production sketch. It shows the control structure and safety order; sensor drivers, calibration values, relay polarity, battery integration, cloud authentication and exact board pins must be added for the selected hardware.

```
#include <Arduino.h>

// Example for a classic ESP32-WROOM-32 DevKit only. Verify every pin
// against the exact board. Do not drive a pump directly from a GPIO.
constexpr uint8_t SOIL_PIN = 34;        // ADC1 input on classic ESP32
constexpr uint8_t PUMP_DRIVER_PIN = 18; // rated external driver/relay
constexpr uint8_t TANK_LOW_SWITCH_PIN = 27; // optional low-water float switch

// Replace after calibration in the selected growing medium.
constexpr int SOIL_RAW_DRY = 3000;
constexpr int SOIL_RAW_WET = 1300;
constexpr float MOISTURE_START_PCT = 35.0f; // demonstration only
constexpr float MOISTURE_STOP_PCT  = 48.0f; // demonstration only
constexpr uint32_t SAMPLE_MS = 2000;
constexpr uint32_t MAX_PUMP_RUN_MS = 15000; // bench-test value only
constexpr uint32_t MIN_COOLDOWN_MS = 60000;

struct Snapshot {
  int soilRaw = -1;
  float soilPct = NAN;
  bool tankAvailable = false;
  bool valid = false;
  uint32_t sampledAt = 0;
};
```

```
enum class IrrigationState { MONITOR, IRRIGATING, FAULT_SAFE };
IrrigationState state = IrrigationState::MONITOR;
Snapshot last;
uint32_t lastSampleAt = 0;
uint32_t pumpStartedAt = 0;
uint32_t lastPumpStoppedAt = 0;

void pumpOff() {
  // Confirm the relay/driver polarity on the bench before connecting pump.
  digitalWrite(PUMP_DRIVER_PIN, LOW);
}

void enterFaultSafe(const char* reason) {
  pumpOff();
  state = IrrigationState::FAULT_SAFE;
  Serial.print("FAULT_SAFE: "); Serial.println(reason);
}

float calibrateMoisturePct(int raw) {
  // Assumes dry raw > wet raw. Clamp only after checking sensor range.
  if (SOIL_RAW_DRY <= SOIL_RAW_WET) return NAN;
  float pct = 100.0f * (float(SOIL_RAW_DRY - raw) /
                        float(SOIL_RAW_DRY - SOIL_RAW_WET));
  return constrain(pct, 0.0f, 100.0f);
}

Snapshot readSnapshot() {
  Snapshot s;
  s.sampledAt = millis();
  s.soilRaw = analogRead(SOIL_PIN);
  s.soilPct = calibrateMoisturePct(s.soilRaw);
  // Example assumes a wired float switch. Replace with a validated
  // ultrasonic tank-level driver if using the proposal's ultrasonic sensor.
  // Change this polarity to match the actual switch and input circuit.
  s.tankAvailable = (digitalRead(TANK_LOW_SWITCH_PIN) == HIGH);
  s.valid = (s.soilRaw >= 0 && s.soilRaw <= 4095 && !isnan(s.soilPct));
  return s;
}

void setup() {
  Serial.begin(115200);
  pinMode(PUMP_DRIVER_PIN, OUTPUT);
  digitalWrite(PUMP_DRIVER_PIN, LOW); // default OFF before other setup
  pinMode(TANK_LOW_SWITCH_PIN, INPUT_PULLUP);
  analogReadResolution(12);
  // Initialise sensor/network modules only after outputs are safe.
}

void loop() {
  const uint32_t now = millis();
  if (now - lastSampleAt >= SAMPLE_MS) {
    lastSampleAt = now;
    last = readSnapshot();

    if (!last.valid) {
      enterFaultSafe("invalid soil sensor reading");
    } else if (!last.tankAvailable) {
      pumpOff();
      if (state == IrrigationState::IRRIGATING) lastPumpStoppedAt = now;
      state = IrrigationState::MONITOR;
      Serial.println("WATER_LOW: pump held OFF");
    } else if (state == IrrigationState::FAULT_SAFE) {
      // Demonstration behaviour: require operator reset / fault-clear logic.
      pumpOff();
    } else if (state == IrrigationState::IRRIGATING) {
      const bool timeout = (now - pumpStartedAt >= MAX_PUMP_RUN_MS);
      const bool targetReached = (last.soilPct >= MOISTURE_STOP_PCT);
      if (timeout || targetReached) {
        pumpOff();
        lastPumpStoppedAt = now;
        state = IrrigationState::MONITOR;
        Serial.println(timeout ? "STOP: max runtime" : "STOP: moisture target");
      }
    } else {
      const bool cooldownComplete =
          (lastPumpStoppedAt == 0 || now - lastPumpStoppedAt >= MIN_COOLDOWN_MS);
      if (last.soilPct <= MOISTURE_START_PCT && cooldownComplete) {
        // A production version also checks SoC, PV power/forecast, crop urgency,
```

```
      // pump current/flow feedback, zone valve state, and all safety interlocks.
      pumpStartedAt = now;
      digitalWrite(PUMP_DRIVER_PIN, HIGH); // verify active polarity first
      state = IrrigationState::IRRIGATING;
      Serial.println("START: moisture below threshold");
    }
  }
  Serial.printf("soil_raw=%d soil_pct=%.1f tank=%d state=%d\n",
                last.soilRaw, last.soilPct, last.tankAvailable, int(state));
}

// Run bounded network/reconnect work and climate-control tasks here.
// Never allow a network request to block pump timeout checks.
}
```

Before hardware use This reference is for a small bench demonstration and is not a complete implementation of the proposal. Verify active-high/active-low relay behaviour, float-switch polarity, moisture calibration, electrical isolation and pump-driver ratings with the pump disconnected first. Add energy checks, zone valves, ultrasonic driver, command authentication and fault-clear policy before full integration.

10. Failure handling and cybersecurity

10.1 Failure-mode policy

Failure / condition	Local reaction	Cloud/app reaction
Wi-Fi or API unavailable	Continue local sensing and safety control; bounded telemetry buffer; reconnect with exponential backoff.	Show last-seen time and stale-data badge; do not claim live status.
Soil sensor invalid/stale	Do not initiate ordinary irrigation based solely on that signal; apply validated crop-specific fallback and/or enter fault-safe state.	Create sensor fault event and maintenance alert.
Tank low / no water	Pump OFF immediately; prevent dry running.	Raise WATER_LOW alert; log affected zone and time.
Pump runtime exceeded	Pump OFF; latch fault until safe recovery/reset policy.	Raise IRRIGATION_TIMEOUT with relevant telemetry.
Battery reserve reached	Stop discretionary heavy loads; keep essential sensing/control alive as energy budget permits.	Report energy-constrained mode; keep irrigation emergency policy explicit.
Forecast missing/stale	Fall back to local PV power, battery reserve and non-forecast local policy.	Mark forecast unavailable; avoid showing fabricated confidence.
Remote command expired/duplicated	Reject expired command; de-duplicate by command ID; still enforce local limits.	Record rejected/duplicate status and reason.
Reboot during watering	Boot with pump OFF; do not automatically resume prior pump command. Re-evaluate from fresh readings.	Report reboot and interrupted action event.
Database/authentication failure	No change to local safety; reject untrusted remote commands.	Log access errors and preserve least privilege.

10.2 Security baseline

- Use TLS/HTTPS for device API communication and validate server certificates; never send device credentials in plain text.
- Give each device its own identity/credential that can be revoked and rotated. Do not embed Firebase Admin/service-account credentials in ESP32 firmware.
- Flutter users authenticate through Firebase Authentication. Use least-privilege security rules scoped to user, greenhouse and role.
- Commands require an authorised role, allowed command type, timestamp/expiry, device scope and audit event.
- Validate all request bodies, sensor units, payload sizes and rates. Record security-relevant failures without logging secrets.
- Keep API keys and secret credentials outside source control. Use environment/secret management in the cloud backend.
- Use an approved manual override policy: duration, role, expiry, visible override status, and automatic return to a safe mode.

11. Verification and performance evaluation

11.1 Test layers

Test level	Example checks	Evidence to retain
Unit tests	Moisture calibration math, hysteresis, command expiry, safe priority, forecast stale logic.	Test cases, expected/actual result, software version.
Bench hardware	Sensor readings with pump disconnected; relay default state; water-level sensor; power monitor scaling.	Calibration notes, meter comparisons, photographs/wiring revision.
Actuator tests	Pump stop time, max runtime, valve sequence, fan switching, shade travel limits.	Timestamped event log and observed hardware result.
Integration tests	ESP32→HTTPS API→database→Flutter; command acknowledgement; cloud unavailable scenario.	Request/response IDs, app screen capture, logs.
Safety/fault injection	Unplug sensor, empty tank, force timeout, interrupt Wi-Fi, reboot mid-cycle.	Pass/fail matrix and recovery behaviour.
AI validation	Baseline versus model; data split by crop cycle/time; compare to agronomist review.	Dataset card, metrics, model version, reviewed recommendations.
Field evaluation	Water volume, PV energy, battery behaviour, crop observations and control versus baseline.	Daily logbook, sensor calibration, experimental conditions and analysis.

11.2 Metrics and definitions

- Water use (L/day and L/kg harvested where relevant); water saving = (baseline use − system use) / baseline use × 100%, only under comparable conditions.
- Energy balance: Wh produced, Wh consumed by each load, battery charge/discharge, minimum observed SoC and energy used per irrigation event.
- Irrigation response: request-to-actuation delay, total watering duration, target threshold attainment, emergency-stop time and timeout frequency.
- Sensor quality: calibration error against an appropriate reference, invalid sample rate, drift and data freshness.
- AI quality: select metrics appropriate to the actual target; include baseline performance, sample count, held-out evaluation and expert review.
- Reliability: uptime, number/duration of faults, successful uploads, missed/duplicate commands and recovery time.

Do not pre-claim outcomes Report “reduced water use by X%” or “improved prediction accuracy by Y%” only after the selected baseline, sample size, conditions and calculation support the claim.

12. Delivery plan, risks and acceptance gates

Phase	Work package	Exit / acceptance gate
P0 — Design freeze	Crop selected; BOM corrected; wiring, power budget, thresholds and data model reviewed.	Supervisor/engineering review signed; no unresolved high-risk power or pin decisions.
P1 — Sensor bench	ESP32 reads/calibrates moisture, temperature/RH, light, water and voltage/current sensors.	Readings have units, limits, validity flags and saved calibration records.
P2 — Local irrigation	One zone first; pump driver, tank interlock, hysteresis, timeout and manual stop.	Pump never runs dry in fault tests; timeout stops pump; disconnected sensor is detected.
P3 — Multi-zone + energy	Zones, battery/PV measurement and energy-aware policy.	Energy constrained, high-solar, critical-crop and no-water scenarios pass.
P4 — Cloud + mobile	API, database, authentication, telemetry, history, command queue and acknowledgements.	A complete telemetry/action round-trip works; local control persists with cloud unavailable.

Phase	Work package	Exit / acceptance gate
P5 — AI advisory	Crop records, agronomic baseline, training/evaluation process and advisory screen.	Every recommendation carries version/evidence and review status; no unsafe dosing.
P6 — Field validation	Run system under controlled conditions, compare baseline and document limitations.	Pass/fail test report, measured energy/water/crop metrics, final build and handover.

12.1 Main project risks

Risk	Mitigation
Scope too large for one project cycle	Limit first release to one crop, one greenhouse, moisture-based multi-zone irrigation, energy-aware scheduler and an advisory nutrition module.
Insufficient AI training data	Start data collection immediately; use a transparent expert-reviewed baseline; make ML claims only after evaluation.
Incorrect energy sizing	Measure pump/fan current, calculate Wh/day, account for concurrent loads and poor-solar days; get electrical design reviewed.
Sensor drift or noisy readings	Calibration, filtering, validity/freshness flags, maintenance schedule and reference measurements.
Cloud or Wi-Fi outages	Local-first logic, retry/backoff, bounded buffer, stale-data indicator and no dependency on cloud for safety.
Unexpected pump/valve actuation	Safe boot defaults, local interlocks, short command expiry, manual emergency stop, command audit and bench tests.
Budget overrun	Reconcile proposal totals, collect supplier quotations and reserve contingency for installation/protection parts.

12.2 Six-month suggested schedule

Month	Primary focus	Evidence
1	Requirements, crop profile, literature, architecture, updated BOM and risk register.	Design review package.
2	Bench sensors, calibration, basic C++ modules and power-system calculation.	Working sensor prototype.
3	Single-zone irrigation, hard safety interlocks, driver wiring and local event log.	Safe local control demonstration.
4	Multi-zone control, PV/battery monitoring, energy scheduler and greenhouse mechanical build.	Energy-aware control demonstration.
5	Cloud API, Firebase, Flutter dashboard, command audit, forecast and AI baseline/data pipeline.	End-to-end connected prototype.
6	Integration, fault injection, field measurements, model evaluation and final documentation.	Evidence-based test report and handover.

Appendix A. Reference implementation patterns

A1. Decision-output contract

Keep the irrigation policy output explicit and auditable. The pure decision function should return a decision object; a separate actuator layer applies it only after checking safety status.

```
enum class IrrigationAction {
  KEEP_OFF,
  START_ZONE,
```

```
  WAIT_FOR_ENERGY,
  STOP_AND_ALERT,
  ENTER_FAULT_SAFE
};

struct IrrigationDecision {
  IrrigationAction action;
  int zoneId;                 // -1 when not applicable
  uint32_t maxRunSeconds;     // must be <= locally configured hard limit
  const char* reasonCode;     // e.g. SOIL_DRY_ENERGY_OK
  bool criticalCropOverride;
};

// Conceptual function signature:
// IrrigationDecision decideIrrigation(const Snapshot& data,
//                                     const CropProfile& crop,
//                                     const EnergyStatus& energy,
//                                     const SafetyStatus& safety,
//                                     const IrrigationHistory& history);
//
// The function must not write GPIO. Unit-test its returned decision first.
```

A2. Example reason codes for analytics

Reason code	Meaning
MOISTURE_OK	Moisture is at/above configured stop threshold.
SOIL_DRY_ENERGY_OK	Soil requires water and energy checks passed.
WAIT_SOLAR_WINDOW	Deferral allowed because crop is not critical and a safe window is expected before deadline.
CROP_CRITICAL_OVERRIDE	Urgent crop need superseded energy optimisation, while electrical safety remained valid.
WATER_LOW	Tank level below low-water limit; pump inhibited.
SENSOR_INVALID	Required measurement missing, stale or outside allowed range.
PUMP_TIMEOUT	Maximum run duration reached.
REMOTE_COMMAND_EXPIRED	Command rejected due to expired validity window.
FORECAST_UNAVAILABLE	Local policy used because forecast is missing/stale.

A3. Pre-deployment checklist

- [ ] Exact ESP32 board/model and pin map confirmed.
- [ ] All sensors calibrated in the real environment; units/ranges documented.
- [ ] Solar, battery and load calculations reviewed; charge controller and protections match the actual system.
- [ ] Pump/valve/fan drivers are rated for actual current and startup/stall conditions.
- [ ] Safe output state verified during boot, brownout, reset, Wi-Fi loss and sensor loss.
- [ ] Tank-empty and maximum-runtime tests pass with pump running on the bench/controlled setup.
- [ ] Remote commands authenticated, scoped, expiring, de-duplicated and audited.
- [ ] Firebase security rules tested with authorised and unauthorised users.
- [ ] Local safety behaviour tested with cloud services disconnected.
- [ ] AI recommendations display evidence/version/approval and cannot initiate unsupported fertilizer dosing.
- [ ] All project claims in the report match the actual implemented and tested functions.

Appendix B. Technical references

Project-specific baseline: IRRIGA-TECH.pdf supplied in this conversation, particularly Chapters Three and Four, the project budget, Bill of Materials and Appendices C–I. This blueprint retains the proposal's intended greenhouse scale and principal subsystems while marking unresolved implementation details as design decisions.

Espressif Arduino-ESP32 documentation. Arduino framework, Wi-Fi and ADC API reference. Verify the API and pin details for the selected board/core version. https://docs.espressif.com/projects/arduino-esp32/en/latest/

Firebase for Flutter. Flutter application integration options. https://firebase.google.com/docs/flutter/

Firebase Realtime Database security. Reference for access controls if Realtime Database is selected instead of Firestore. https://firebase.google.com/docs/database/security

Firebase Cloud Functions. Cloud-hosted backend logic and Python support; confirm current runtime/deployment requirements before implementation. https://firebase.google.com/docs/functions/

Firebase Authentication for Flutter. User authentication setup and authorised application access. https://firebase.google.com/docs/auth/flutter/start

Open-Meteo GFS API solar radiation variables. Example forecast source offering solar radiation variables; validate data availability, licensing and terms for the intended use. https://open-meteo.com/en/docs/gfs-api

Appendix C. Project-manager kickoff checklist

Use this as the first execution gate. Do not purchase the full bill of materials or connect pump power until the design decisions and safety tests below have owners and evidence.

1. Confirm the first crop, growing medium, number of zones, irrigation method and which agronomist or agricultural specialist will review crop-specific thresholds.
2. Freeze the exact ESP32 board, sensor models, pump model, valve types, driver modules, battery chemistry, panel rating and charge-controller model; update the bill of materials to match.
3. Measure pump/fan startup and running current, estimate daily operating hours, calculate Wh/day, and review panel, battery, charge-controller, fuse and cable sizing with a qualified electrical reviewer.
4. Build the sensor bench with pump disconnected; record raw values, reference measurements, calibration equations, valid ranges and stale-data timeouts.
5. Implement the local SafetyManager and one irrigation zone first; demonstrate tank-empty protection, sensor-loss handling, manual stop, maximum runtime and safe reboot before cloud integration.
6. Create the firmware repository, database schema, configuration/version policy, event/reason-code list, test log and issue register before parallel software work begins.
7. Create a Firebase development environment with authentication and least-privilege rules; verify that unauthorized users cannot read private farm data or issue device commands.
8. Only after local tests pass, integrate telemetry, cloud recommendations and Flutter controls. Start AI as a reviewed advisory service until the dataset and evaluation results justify stronger claims.